

PROJECT REPORT

Script-Controlled ACL – Restrict Record Access Based on Field Value

Platform: ServiceNow **Project Type:** Security / Access Control Micro Project **Duration:** 3 hours

1. INTRODUCTION

1.1 Project Overview

This project demonstrates record-level security in ServiceNow using Access Control Lists (ACLs). A custom **Institution Details** table is created to store student and faculty information. Access to these records is controlled through user roles, a Branch field, ACL data conditions, and server-side ACL scripting. The documented scenario focuses on restricting access to EEE branch records while retaining full administrator access.

The project implements four record operations: **READ, CREATE, WRITE, and DELETE**. The custom roles **bb1, bb2, bb3, and bb4** are associated with these operations. Verification is performed by impersonating users and opening the Student Records / Institution Details list and forms.

1.2 Purpose

The purpose is to provide a practical understanding of how ServiceNow authorization can be implemented without a separate application server. The project shows how roles and ACL rules can be used to protect records from unauthorized viewing, creation, modification, and deletion.

2. IDEATION PHASE

2.1 Problem Statement

Institution or student records may contain information that should not be visible or modifiable by every ServiceNow user. A generic role-only permission model may not be sufficient when access must be restricted at record level. The project therefore addresses the need to control which users can access institution records based on configured roles and the Branch value.

2.2 Empathy Map Canvas

Perspective	Project Interpretation
User / Says	Needs access only to records relevant to the permitted branch and role.
User / Thinks	Sensitive records should not be visible or editable without authorization.
User / Does	Opens lists/forms and performs permitted record operations.
User / Feels	Expects predictable and secure access behavior.
Administrator	Requires unrestricted administrative access for management and verification.

2.3 Brainstorming

- Use ServiceNow roles to represent different permission levels.
- Use a Branch choice field to distinguish ECE, EEE, and CSE records.
- Use a scripted READ ACL for explicit access decisions.
- Use separate CREATE, WRITE, and DELETE ACLs for operation-level control.
- Use impersonation to validate behavior for different users.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User Activity	Security Check
Login	User signs in to ServiceNow.	ServiceNow authentication.
Navigate	User opens Institution Details records.	ACL evaluation begins.
View	User opens list / record.	READ ACL and configured condition are evaluated.
Create	User selects New and enters data.	CREATE ACL checks bb2.
Edit	User modifies an existing record.	WRITE ACL checks bb3.
Delete	User removes a record.	DELETE ACL checks bb4.
Admin	Administrator opens records.	READ script explicitly allows admin.

3.2 Solution Requirement

- **Functional:** Create the u_institution_details table and required fields.
- **Security:** Create bb1–bb4 roles and operation-specific ACLs.
- **Read control:** Configure the documented EEE data condition and scripted role checks.
- **Operations:** Control create, write, and delete through bb2, bb3, and bb4.
- **Validation:** Verify results using user impersonation and list/form access.

3.3 Data Flow Diagram

Conceptual flow: User → ServiceNow Authentication → User Role Evaluation → ACL Evaluation → Institution Details Table → Allow / Deny Record Operation.

Input	Processing	Output
User request	ServiceNow identifies user and roles.	Authenticated request
Read request	READ ACL evaluates admin / bb1 and configured branch condition.	Record access allowed or denied
Create request	CREATE ACL checks bb2.	New record allowed or denied
Write request	WRITE ACL checks bb3.	Edit allowed or denied
Delete request	DELETE ACL checks bb4.	Deletion allowed or denied

3.4 Technology Stack

Technology / Feature	Use in Project
ServiceNow Platform	Application, user, table, form and security platform.
ServiceNow ACL	Record-level authorization.
JavaScript / ACL Script	Server-side scripted READ decision.
ServiceNow Roles	bb1, bb2, bb3, bb4 and admin.
Custom Table	u_institution_details.
Impersonation	Security verification and testing.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The solution directly maps the security problem to ServiceNow-native controls. The Institution Details table provides the protected data resource, roles define operation permissions, the Branch field supplies record information, and ACLs enforce access decisions. This avoids introducing a separate authorization service for the documented use case.

4.2 Proposed Solution

Create **u_institution_details** with Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number, and Description. Create roles **bb1**, **bb2**, **bb3**, **bb4**. Configure a READ ACL with the documented EEE data condition and script. Configure separate CREATE, WRITE, and DELETE ACLs using bb2, bb3, and bb4.

ACL	Role	Condition / Logic	Expected Capability
READ	bb1	Branch is EEE; script allows admin and bb1.	View permitted records
CREATE	bb2	No data condition documented.	Create records
WRITE	bb3	No data condition documented.	Edit records
DELETE	bb4	No data condition documented.	Delete records

4.3 Solution Architecture

Users → Roles (bb1–bb4) → ServiceNow ACL Engine → u_institution_details → Record Operation. The READ ACL additionally evaluates the documented data condition and server-side script. Administrators receive an explicit true result in the documented script.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase	Activity	Deliverable
1	Create test user and roles.	EEE User; bb1–bb4 roles
2	Create custom table and fields.	u_institution_details
3	Create sample records.	ECE / EEE / CSE records
4	Configure READ ACL.	Role + condition + script
5	Configure CREATE ACL.	bb2 authorization
6	Configure WRITE ACL.	bb3 authorization
7	Configure DELETE ACL.	bb4 authorization
8	Impersonate and verify.	Access validation results

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The source project specifies functional verification through impersonation but does not provide measured response-time, throughput, load, or stress-test results. Therefore, no numerical performance claim is made here. For a production extension, ACL evaluation time and list-query behavior can be measured under representative record volumes.

Test	Expected Result
bb1 user opens list	Can view the documented EEE records.
User without required access	Cannot view records.
Admin opens list	Can view all records.
bb1 + bb2	View permitted records and use New/Create.
bb1 + bb2 + bb3	View and edit permitted records.
bb1 + bb2 + bb3 + bb4	View, create, edit and delete as configured.

7. RESULTS

7.1 Output Screenshots

The supplied project source does not contain actual ServiceNow screenshots. To keep the report factual, screenshot placeholders are provided below for insertion from the implemented instance.

Screenshot 1 – Institution Details table / list showing ECE, EEE and CSE records

Screenshot 2 – READ ACL configuration with Branch = EEE and script

Screenshot 3 – Impersonated user access verification / list view

8. ADVANTAGES & DISADVANTAGES

Advantages: Native ServiceNow implementation; operation-specific access control; role-based authorization; record-level security; administrator override in the documented script; straightforward impersonation-based verification.

Disadvantages / limitations: The source does not provide automated testing, performance measurements, deployment procedures, screenshots, or external API documentation. The documented READ script checks the bb1 role but does not itself inspect a user's branch attribute.

9. CONCLUSION

The project demonstrates a complete ServiceNow ACL security scenario covering READ, CREATE, WRITE, and DELETE operations. It shows how custom roles, record conditions, and scripted authorization can be combined to control access to institution records. The documented outcome is a practical understanding of record-level security and ACL-based protection of sensitive data.

10. FUTURE SCOPE

- Introduce automated ACL test cases for all role and operation combinations.
- Formalize user-to-branch mapping if branch-specific authorization is required.
- Add field-level ACLs for particularly sensitive attributes.
- Add security auditing and monitoring for record changes.
- Document update-set/application deployment and rollback procedures.
- Measure ACL and list performance using representative data volumes.

11. APPENDIX

Source Code (if any): The project contains the documented server-side READ ACL script shown below.

```
(function () {  
  if (gs.hasRole('admin')) { return true; }  
  if (gs.hasRole('bb1')) { return true; }  
  return false;  
})();
```

Dataset Link: Not supplied. The source instructs creation of sample records with ECE, EEE, and CSE branch values.

GitHub & Project Demo Link: Not supplied in the project source.

Report basis: Final Report Format + Script-Controlled ACL project specification. Project-specific details are kept aligned with the supplied materials.